

Addressing a Bias in Evaluating of Student Self-Explanations of Worked Programming Examples

Arun-Balajee Lekshmi-Narayanan, Xiang Lorraine Li, Peter Brusilovsky

arl122@pitt.edu,

Intelligent Systems Program,
School of Computing and Information
University of Pittsburgh

Abstract

Worked examples are step-by-step solutions to problems in a specific domain, offered to students to acquire domain-specific problem-solving skills. The power of worked examples could be magnified by combining them with self-explanations, which ask students to explain rather than passively study each problem-solving step. The main challenge of this approach is assessing the correctness of the student's explanations. In the current approach, student explanations are judged by their semantic similarity to an explanation provided by an instructor or domain expert. However, recent studies of example explanations in the domain of programming demonstrated that many students express themselves very differently from domain experts. In this situation, a traditional semantic similarity approach might introduce bias against students who correctly explain worked examples but are considerably different from expert explanations. In this paper, we use a recently published dataset to compare several explanation-assessment approaches based on semantic similarity with alternative approaches based on direct Large Language Model (LLM) prompting. Our results show that the use of LLMs enables worked example systems that follow an active learning approach to reduce bias in evaluating example explanations.

Introduction

In the domain of programming, worked examples (WE) are offered as “explained code” where the lines or fragments of a program (code) solving a problem are augmented with expert explanations (Linn, 1992; Brusilovsky, Hsiao, and Yudelson, 2008). Originally, the main source of explained programming WEs were textbooks (Deitel and Deitel, 1994; Kelley and Pohl, 1995), however, nowadays students more frequently learn from worked examples while using “code-casts”, narrated videos where experts explain code as they write it (Sharrock et al., 2017) or example exploring systems (Hosseini et al., 2020) where students can examine WE line-by-line while reading expert explanations.

Although these interactive tools demonstrated their positive impact on learning in several studies (Brusilovsky, Hsiao, and Yudelson, 2008; Hosseini et al., 2020), reading or listening is classified as *passive learning*, which is not the

most efficient way to learn. Summarizing multiple years of learning science research, ICAP framework (Chi and Wylie, 2014) argues that active learning promotes greater comprehension than passive learning. As pointed out by (Koedinger et al., 2015), “Learning is not a spectator sport”: to learn more efficiently, students need to be actively engaged in *doing* rather than passively reading or listening.

An efficient way to turn learning from examples from passive to active is *self-explanations*, an approach in which learners are prompted to explain WE themselves rather than reading expert explanations (Chi et al., 1989, 1994). Learning programming was one of the first domains in which the power of self-explanations has been explored (Bielaczyc, Pirolli, and Brown, 1995). Currently, this approach is used in both assessment and study contexts. In an *assessment context*, instructors include code explanation questions in home assignments and exams. In a *study context*, students are encouraged to self-explain code-based WE either by adding explanatory comments to WE code (Garces et al., 2023) or by writing code explanations to their workbooks (Vihavainen, Miller, and Settle, 2015). Unfortunately, neither of these approaches to engage students in self-explanations is satisfactory. Self-explanation questions are among the hardest for instructors to grade and provide corrective feedback, so they are used quite rarely. In contrast, using self-explanations in a study mode without assessment and feedback avoids the problem of instructor overload, but it leaves the learning opportunity incomplete and can lead to forming dangerous misconceptions.

A recent breakthrough in natural language processing has spurred a stream of pioneering research on the automatic evaluation of student self-explanations (Rus et al., 2021; Chapagain and Tamang, 2022; Oli et al., 2024c). In this work, the correctness of student self-explanations was assessed by comparing student self-explanations with expert explanations for the same lines or fragments of code using a semantic similarity approach, such as the Deep Tutor Banjade et al. (2016). This new technology has been evaluated in several studies that have shown positive results (Chapagain and Tamang, 2022; Oli et al., 2024c). Recently, a dataset from one of these studies has been released by the project team (Chapagain et al., 2025).

This paper was motivated recent studies comparing student and expert code explanations. In one study, student ex-

planations were compared with expert and Large Language Model (LLM) explanations using several metrics (Lekshmi-Narayanan et al., 2024). The study revealed that, overall, student explanations differ considerably from expert explanations in lexical density, length, vocabulary, and readability. Another study (Lekshmi-Narayanan and Brusilovsky, 2024) demonstrated that explanations for the same code lines written by different students differ considerably in their semantic similarity to expert explanations for these lines; moreover, student explanations could be very different from experts', yet correct. Taken together, these studies suggest that an approach to evaluating self-explanations based on semantic similarity might be biased against students who express themselves very differently from experts. In this paper, we explore whether this potential bias in evaluating code explanations using semantic similarity approaches exists and whether it could be reduced by alternative assessment approaches based on LLM prompting. Our primary research question is the following: *could students who write explanations in a way that is less similar to expert explanations be assessed more fairly using LLMs rather than semantic similarity approaches?*

Related Work

Evaluating free-form student responses, such as open-ended questions, essays, and self-explanations, has long been a challenge in the field of intelligent educational systems. A significant breakthrough in this area was the introduction of semantic-similarity approaches to compare student responses with instructor-provided model answers. The first evaluation technology that could be called semantic similarity was Latent Semantic Analysis (LSA), first used in AutoTutor, the oldest dialog tutoring system (Graesser et al., 2000), and in many other first-generation projects (Wild et al., 2005).

Following LSA, the researchers explored a sequence of increasingly powerful probabilistic approaches, including topic models and Latent Dirichlet Allocation (LDA). A representative example of this generation is the SEMILAR toolkit (Rus et al., 2013b), specifically focused on evaluating student responses. Its similarity function was an ensemble model that processed the text using the Quadratic Assignment Problem and weighted LDA topic modeling to generate an objective score of semantic similarity between student and expert text explanations.

The next generation of similarity approaches was based on deep learning technologies. An example of this approach is a transformer extensions to SEMILAR (Chapagain and Tamang, 2022), which utilized sentence embeddings created by SentenceBERT (Reimers and Gurevych, 2019). SentenceBERT provides models such as MiniLM to generate sentence embeddings for student and expert responses. As this may deviate from the original goal of calculating the semantic scoring of student responses, researchers (Chapagain and Tamang, 2022) separately trained the support vector regression model to generate a final objective score, choosing between the statistical SEMILAR model and deep neural network embeddings (Reimers and Gurevych, 2019).

The most recent research explored the application of LLMs to assess the correctness of student explanations. For example, Oli et al. (2024a) investigated a finetuned open-source LLM with preference optimization to generate feedback for students in a dialog-based intelligent tutoring system. In their interface, the students could make several attempts to explain the code line, engaging in a dialogue with the tutoring system (Oli et al., 2024c). A different LLM-based approach was suggested by Denny et al. (2024). In this project, the students had to demonstrate their code comprehension skills by explaining the code of small programs. To assess the correctness of these explanations, they were submitted to the LLM as problem statements to generate code to solve the problems. They compared the behavior of the original code and the generated code. The equivalent behavior indicated that the student's code explanations were correct.

Inspired by these successful attempts, we explored the application of LLMs to a more challenging case: line-by-line code explanations. In contrast to the approaches reviewed above, our approach, presented in this paper, focused on prompting LLM directly to assess the correctness of student explanations.

Dataset

To answer our research question, we used the recently released SelfCode2.0 dataset (Chapagain et al., 2025), which includes explanations produced by 60 students and 2 experts for individual lines of four code examples. In the dataset, each student's explanation is labeled by annotators as correct or incorrect. In total, the dataset included 3019 pairs of student and expert explanations for the same code lines. For our study, we used a subset of these pairs (1854 out of 3019) in which both a student and an expert explained the line using a single statement. This selection was made to facilitate the exploration of modern semantic-similarity approaches that focus on complete sentences. In these 1854 pairs, 1794 student explanations were labeled as correct (positive examples) and 60 as incorrect negative examples.

To examine the differences between student and expert explanations in the same pair, we used the Deep Tutor similarity approach developed by the authors of the data set Chapagain et al. (2023) (Figure 1). Each column in this figure represents a code line in one of the four WE in the data set, and each dot represents a normalized similarity between student and expert explanations in a specific explanation pair for this line. Correct explanations are shown as blue dots and incorrect ones as red dots. The analysis confirmed both observations of earlier research: (1) student explanations for the same line could differ broadly in their similarity to expert explanations, with similarity scores ranging from below 0.2 to above 0.9; (2) many student correct explanations are very different from expert explanations for the same line. In particular, the difference between many correct and expert explanations was below 0.3, which was the threshold for classifying the explanation as incorrect in the study that collected the data set. These data confirmed the original concern that evaluation approaches based on semantic similarity might not be fair to students who correctly explain code lines

but differ significantly from experts, especially unjustly penalizing their correct answers. The dataset provides data to understand *why* some students explain code very differently from experts - they might be “non-traditional” students from a different demographic, speak a different native language, or just ignored the instructor’s explanations during the lecture. However, the dataset provides sufficient data to explore how fair different assessment approaches evaluate these students, which is the focus of our paper.

Method

Large Language Model (LLM) Based Evaluation Approaches

Our LLM-based approaches for evaluating student explanations are based on treating an LLM as an “oracle” that directly predicts the *correctness* of student explanations. We prompt the LLM with the context that included the problem statement of the explained WE, the code line to explain along with the line number, and the associated student explanations from the data set (Figure 2). Before finalizing the prompt, we manually reviewed 200 student explanations across the similarity range. We observed that some students focused on explaining the line’s behavior (behavior-focused explanations). In contrast, others concentrate on why the line was used to achieve the program’s goal (construction-focused explanations). To avoid unfairly assessing students who favored one of these explanation styles, we developed 3 versions of the LLM prompt, each with a different definition of correctness at the beginning. In all prompts, we use the same section headings in capital letters, such as “PROGRAM DESCRIPTION”, “LINE NUMBER” and so on (Figure 2). We used the prompts with GPT-3.5-Turbo-16k for the automated assessment task.

No Definition We do not define “correctness” to LLM for evaluation (marked (a) in the Figure 2).

Construction We defined correctness by focusing on the construction-based approach to explain a code line, i.e., the role of the line in solving the programming problem (marked (b) in Figure 2).

Code Behavior Prompt We define correctness by focusing on the behavior-based approach to explain a code line, i.e., the result of executing this line (marked (c) in Figure 2).

Semantic Similarity Based Evaluation Approaches

We used three popular versions of semantic modeling approaches:

- the original Deep-Tutor (DT) toolkit that adopted the algorithm described to evaluate correctness in the DT-GRADE corpus (Banjade et al., 2016) and was used in process of data collection of our dataset (Chapagain et al., 2025). The method generates similarity scores using Support Vector Regression with a feature set combining SentenceBERT and DeepTutor similarity scores (Rus et al., 2013a).

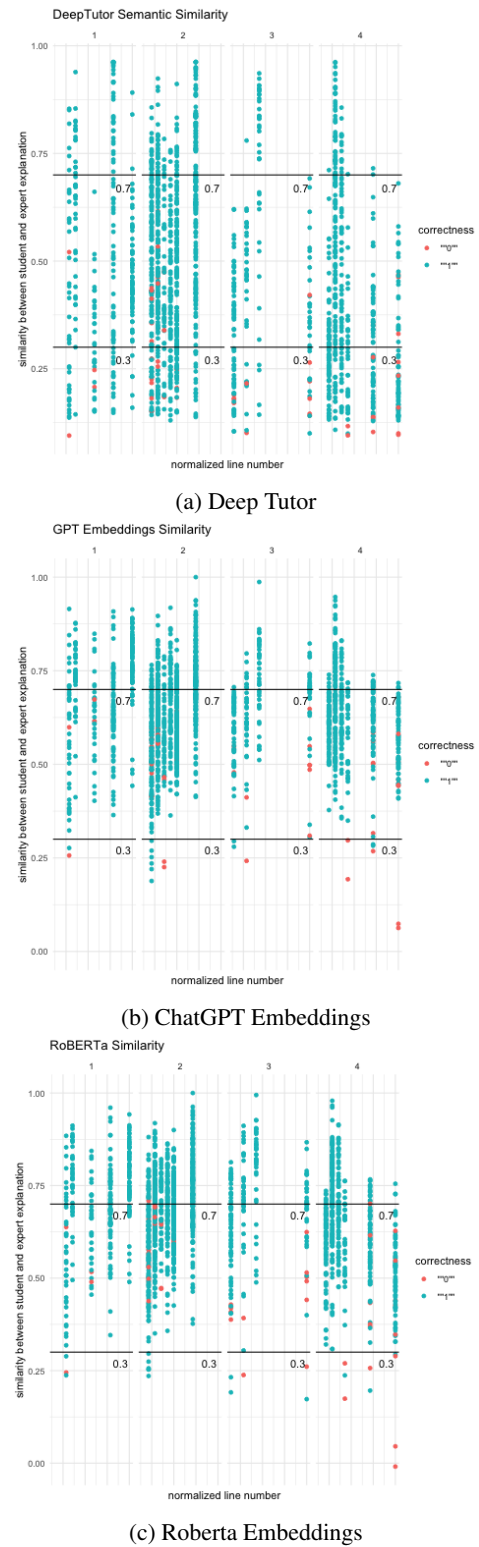


Figure 1: The students’ explanations similarity distribution with expert explanations across line numbers. The plot is grouped by problem numbers 1 through 4.

Threshold	0.00	0.05	0.10	0.15	0.20	0.25	0.30	0.35	0.40	0.45	0.50	0.55	0.60	0.65	0.70	0.75	0.80	0.85	0.90	0.95	1.00
RoBERTa	0.67	0.67	0.67	0.67	0.68	0.68	0.70	0.70	0.71	0.73	0.72	0.73	0.71	0.69	0.60	0.45	0.27	0.13	0.04	0.01	0.00
Deep Tutor	0.67	0.67	0.68	0.70	0.72	0.72	0.71	0.70	0.65	0.60	0.58	0.52	0.45	0.38	0.28	0.21	0.14	0.08	0.05	0.02	0.00
GPT-SE	0.67	0.67	0.67	0.67	0.68	0.69	0.70	0.70	0.70	0.70	0.72	0.73	0.74	0.64	0.48	0.30	0.15	0.06	0.02	0.00	0.00

Table 1: Selecting Model thresholds for RoBERTa, GPT embeddings and Deep Tutor Semantic Similarity Scores using their **weighted** F1 Scores.

Determine the correctness of the student's explanation based on a given source code, specific line number.
(a) Set "Correctness" to 1 if the student's explanation is correct and 0 if it is incorrect.
OR
(b) "Correctness" A correct EXPLANATION explains why the line is used while implementing this program given the PROGRAM DESCRIPTION and SOURCE CODE below.
OR
(c) "Correctness" A correct STUDENT EXPLANATION explains the code behavior using the code syntax.
ONLY set "Correctness". DO NOT PROVIDE REASON.
PROGRAM DESCRIPTION
{program desc}
SOURCE CODE
{source code}
LINE NUMBER
{line num}
STUDENT EXPLANATION
{student explanation}
CODE
{line content}
Correctness:
width

Figure 2: Prompting LLM to evaluate students' explanations. "OR" notation is used in this to indicate only one of the 3 possibilities were used to define "Correctness" to the model.

- RoBERTa (Liu et al., 2019), a version of BERT with appropriate modifications for Next Sentence Prediction tasks, available through sentence-transformers¹
- GPT Sentence Embeddings (GPT-SE)², available as an API which creates a vector representation for any form of text, either broken down into sentences, individual tokens, or entire paragraphs using the latest LLM from ChatGPT-family of models from OpenAI

For each approach, we computed semantic similarity scores between 0 and 1 for each pair of student and expert explanations. Deep-Tutor is designed to return this score directly. To obtain a semantic similarity score using RoBERTa or GPT-SE, we generated sentence embeddings for all student and expert explanations. We calculated the cosine similarity between the embeddings for student and expert explanations in the same pair. The distribution of similarity scores produced by these three approaches for each code line in the dataset is shown in Figure 1.

To assess student explanations as correct or incorrect, these approaches typically employ a decision threshold.

¹<https://huggingface.co/sentence-transformers/all-roberta-large-v1>

²<https://platform.openai.com/docs/guides/embeddings>

Student responses with similarity above the threshold are considered accurate, while those below the threshold are deemed faulty. For example, in a Deep Tutor-based study used to collect data for the dataset used in this paper, the student answers with similarity less than 0.3 were assessed as incorrect.

In our study, we intentionally placed similarity-based approaches in the most favorable position by determining the optimal decision threshold for each of the three methods using the dataset's correct/incorrect labels. To do this, we iterated over a range of thresholds from 0 to 1 with a step size of 0.05 and selected the threshold that achieved the best F1 score, a harmonic mean of precision and recall. To account for the class imbalance (there were about 30 times more explanations labeled as correct than incorrect), we calculated F1 using upweighted metrics for measures associated with the negative samples, namely, False Positives (FP) and True Negatives (TN). The upweighting was done using the ratio of positive to negative samples in the dataset (1794/60), which was approximately 30. To illustrate the process, we plotted lines representing the F1 scores for each approach at multiple threshold points in Table 1. Using this approach, we determined that the optimal F1 values are 0.5 for Roberta, 0.6 for GPT Embeddings, and 0.2 for Deep Tutor. These thresholds were used to obtain correctness predictions for the similarity approaches in the study presented below.

Results

Performance Comparison

We started the analysis by comparing the overall performance of the six evaluated approaches on the whole set of student-expert pairs. In this analysis, the correct predictions generated by each approach for each pair were compared with the label (correct/incorrect) assigned by the annotators to that pair in the dataset. For each approach, we calculated the True Positives (TP), False Positives (FP), False Negatives (FN), and True Negatives (TN). TP cases are those in which students write correct explanations, and the evaluation algorithm judges them as correct. FN cases are those in which the student's answers are accurate, but the algorithm judges them as incorrect. Using these key parameters, we generated F1 and accuracy scores (note that, in this evaluation, we used real data without upweighting). The results are shown on the left side of Table 3. As the data shows, the LLM-Behavior prompting emerges as the clear winner with the best F1 and Accuracy, while LLM-NoDef follows it closely, lagging just a bit in Accuracy. Among

Dataset	Correct	Incorrect	Total
Total	1794	60	1854
Top-Third Similar	741	1	742
Bottom-Third Similar	687	54	741

Table 2: Splits for counts of ground truth Top and Bottom third Correct / Incorrect explanations from SelfCode 2.0 dataset separated using the METEOR score similarity taken between the student and expert explanations.

the similarity approaches, Deep Tutor and RoBERTa perform equally well, but are notably behind LLM-Behavior and LLM-NoDef. The remaining two approaches lag behind the leaders.

Fairness Comparison

To assess the fairness of the evaluated approaches in judging explanations of students who explain code lines differently from experts, we extracted two subsets of the original dataset representing the most similar and least similar students. We extracted METEOR-based similarity scores for each student and each expert, calculated as the average of the similarity scores across all student-expert pairs associated with a student. We selected the METEOR score as the basis for the split because it was used in all previous studies reviewed above and because it focuses on word similarity, which is the most critical aspect distinguishing dissimilar students. Unlike other semantic similarity scores, METEOR does not give any of the evaluated approaches an unfair advantage. The “most similar” subset of the dataset included data from the top 1/3 students, ranked by similarity to experts, and the “least similar” subset included data from the bottom 1/3 students. We present data for these subsets in Table 2.

Following the isolation of the top and bottom cohorts, we compared the performance of the six approaches on each of the extracted subsets. In addition to the composite metrics like F1, in this evaluation, we paid specific attention to the ability of each approach to minimize False Negatives, the cases where a student’s correct explanation is judged as incorrect. This misjudgement, which led to high levels of frustration, was our original concern about the application of the semantic similarity approach to evaluation.

The results for the most similar and least similar subsets of students are shown on the right side of Table 3. As the data show, Deep Tutor, RoBERTa, LLM-NoDef, and LLM-Behavior approaches produce very few False Negatives on the “most similar” subset. At the same time, GPT-SE and LLM-Constr perform much worse in this respect. However, the situation dramatically changes for the “least similar” subset. Here, all semantic similarity approaches, especially GPT-SE, perform much worse than on the “most similar” dataset. In contrast, all LLM-based approaches, including LLM-Constr, produce about the same number of False Negatives on both subsets. As a result of this notable deterioration in semantic similarity approaches on the “most similar” subset, LLM-NoDef and LLM-Behavior emerge as clear winners in terms of fairness, and even LLM-Constr becomes

comparable to the Deep Tutor and RoBERTa, which outperformed it on the “most similar” dataset. These results show that our concern about the unfair performance of semantic similarity approaches is fully justified, and that LLM-based approaches could offer a much fairer alternative.

Beyond the fairness aspect of the evaluation, LLM-NoDef and LLM-Behavior perform very well on both subsets of our dataset. Their F1 and accuracy on the “most similar” subset are comparable to those of the best semantic similarity approaches, Deep Tutor and RoBERTa, while on the “least similar” subset, they outperform them by a large margin. The excellent performance of LLM-NoDef and LLM-Behavior is consistent with the LLM-as-a-judge studies (Li et al., 2024; Zhang et al., 2024), pointing to the benefits of the non-linearity and free-text prompting that allows better capture of the context.

Discussion and Conclusion

Our paper was motivated by the concern that popular semantic similarity approaches that evaluate the correctness of student explanations by comparing them to the model explanations provided by instructors might be biased against students who express themselves differently from instructors. This concern was supported by past research that demonstrated a notable difference between student explanations and expert explanations overall (Lekshmi-Narayanan et al., 2024), and a broad distribution of student answers by their similarity to expert answers (Lekshmi-Narayanan and Brusilovsky, 2024). In our paper, we attempted to examine this concern using a recently released dataset of labeled student explanations (Chapagain et al., 2025). We also assessed whether evaluation approaches based on LLM prompting could offer a fairer alternative to semantic similarity approaches.

Our results confirmed that, across all three explored semantic similarity approaches, students differ widely in the similarity of their explanations to expert explanations. Some students explain code lines in the same way as instructors, while others describe them very differently, as assessed by semantic similarity. We also observed that the vast majority of students who explained code lines differently from instructors produced explanations judged as correct by the dataset annotators. These data, shown on (Figure 1), highlighted two problems in the traditional way of evaluating student explanations by combining a similarity scoring with a decision threshold to assess an explanation as correct or incorrect. First, it is tough to set a threshold that reliably distinguishes correct from wrong answers. Second, for any meaningful threshold, students who explain differently from the instructors will be unfairly misjudged. In this situation, a non-linear LLM-based evaluation approach could offer an advantage.

In our evaluation, we placed three evaluated semantic similarity approaches in the most favorable position by finding the optimal threshold using the labeled dataset. However, when compared to LLM-based approaches on the same dataset, all semantic similarity approaches performed worse than the two best-performing LLM-based approaches. Moreover, a fairness evaluation performed by isolating the

Dataset Model	Overall						Top Similar						Bottom Similar					
	F1	Acc	TP	FP	FN	TN	F1	Acc	TP	FP	FN	TN	F1	Acc	TP	FP	FN	TN
Deep Tutor	0.94	0.89	1619	37	175	23	0.99	0.99	733	1	8	0	0.86	0.77	545	31	142	23
Roberta	0.94	0.89	1615	32	179	28	0.99	0.97	723	1	18	0	0.88	0.8	563	27	124	27
GPT-SE	0.79	0.66	1168	7	626	53	0.92	0.86	634	0	107	1	0.61	0.47	302	6	385	48
LLM-Behavior	0.98	0.96	1774	46	20	14	1.00	0.99	735	1	6	0	0.96	0.93	677	44	10	10
LLM-NoDef	0.98	0.96	1772	59	22	1	0.99	0.98	730	1	11	0	0.96	0.92	683	53	4	1
LLM-Constr.	0.89	0.8	1457	29	337	31	0.89	0.8	593	0	148	1	0.87	0.77	549	29	138	25

Table 3: Comparing All Methods on Correctness for Top & Bottom Similar Students Separated using METEOR similarity scores between expert and student explanations

most similar and least similar subsets of students in the dataset demonstrated that the performance of semantic similarity approaches notably deteriorated as one moved from most similar to least similar students. In contrast, LLM-based approaches performed similarly across both subsets, treating “least similar” students fairly and producing a much smaller number of false negatives, i.e., assessing students’ correct explanations as incorrect.

The problem with using similarity-based approaches is that the choice of threshold plays a key role in the final prediction of student correctness, reducing the credit for the effort students put into writing these explanations. Using nonlinear LLM approaches allows us to capture more information and features from the student explanations that are not categorized solely as lexical or semantic, but instead as latent-space representations that could have meaning or intent similar to that of the expert. LLMs are better at this latent space representation than the text embeddings on which they are based, as discussed in the context of using LLMs as judges (Li et al., 2024; Zhang et al., 2024).

Limitations

Our work has several limitations. The number of negative examples available in the dataset. For both student explanations that are similar or dissimilar from experts, the dataset has 70 marked incorrect examples. In a future study, we will collect additional examples of student explanations with more diversity, and our system will mark them as incorrect for training set constructions. Other approaches we will explore will use generative AI to simulate incorrect examples for explanations based on the popular student misconceptions in the dataset.

Another limitation is the lack of comparison with a few more state-of-the-art open-source models. As noted by Koutchene and colleagues (Koutchene et al., 2024), they offer capabilities for feedback generation and use as a judge in education. Additionally, we only tried a few prompt engineering techniques; our follow-up work will explore the use of prompt optimization (Zhou et al., 2020; Das et al., 2025) and preference optimization (Oli et al., 2024a) for feedback generation and use as a judge.

Further, using LLMs for automated evaluation was also applied in the context of automatic assessment of student

explanations (Oli et al., 2024b), where the autograding system prioritizes sensemaking. The approach was to prompt LLMs to generate similarity scores between expert and student explanations, similar to the 1–5 scale ratings in the SelfCode (Chapagain et al., 2023) corpus, where humans compare students’ explanations with experts’. The authors found that using Chain-of-Thought prompting with LLMs achieved the highest correlation between LLM-generated similarity scores and human ratings. Our work used LLMs to predict the correctness of student explanations instead of measuring the similarity between expert and student explanations.

In our follow-up work, we will implement LLM-based evaluations to provide feedback on concepts students miss in their explanations, either by comparing them with the expert or by encouraging them to consider them when writing explanations for a line of code.

References

- Banjade, R.; Maharjan, N.; Niraula, N. B.; Gautam, D.; Samei, B.; and Rus, V. 2016. Evaluation dataset (dt-grade) and word weighting approach towards constructed short answers assessment in tutorial dialogue context. In *Proceedings of the 11th workshop on innovative use of NLP for building educational applications*, 182–187.
- Bielaczyc, K.; Pirolli, P. L.; and Brown, A. L. 1995. Training in self-explanation and self-regulation strategies: Investigating the effects of knowledge acquisition activities on problem solving. *Cognition and Instruction* 13(2):221–252.
- Brusilovsky, P.; Hsiao, I.-H.; and Yudelson, M. 2008. Annotated program examples as first class objects in an educational digital library. In *Joint Conference on Digital Libraries, JCDL 2008*, 337–340.
- Chapagain, J., and Tamang, L. 2022. Automated assessment of student self-explanation during source code comprehension. In *The international FLAIRS conference proceedings*, volume 35.
- Chapagain, J.; Risha, Z.; Banjade, R.; Oli, P.; Tamang, L.; Brusilovsky, P.; and Rus, V. 2023. Selfcode: An annotated corpus and a model for automated assessment of self-explanation during source code comprehension. In

- The International FLAIRS Conference Proceedings*, volume 36.
- Chapagain, J.; Narayanan, A. B. L.; Akhuseyinoglu, K.; Brusilovsky, P.; and Rus, V. 2025. Selfcode 2.0: An annotated corpus of student and expert line-by-line explanations of code examples for automated assessment. *The International FLAIRS Conference Proceedings* 38(1).
- Chi, M. T., and Wylie, R. 2014. The ICAP framework: Linking cognitive engagement to active learning outcomes. *Educational psychologist* 49(4):219–243.
- Chi, M. T. H.; Bassok, M.; Lewis, M. W.; Reimann, P.; and Glaser, R. 1989. Self-explanations: How students study and use examples to solve problems. *Cognitive Science* 13:145–182.
- Chi, M. T. H.; De Leeuw, N.; Chiu, M.-H.; and Lavancher, C. 1994. Eliciting self-explanations improves understanding. *Cognitive Science Volume 18, Issue 3, July–September, Pages* 18(3):439–477.
- Das, S. S. S.; Kamoi, R.; Pang, B.; Zhang, Y.; Xiong, C.; and Zhang, R. 2025. Greater: Gradients over reasoning makes smaller language models strong prompt optimizers. In *The Thirteenth International Conference on Learning Representations*.
- Deitel, H. M., and Deitel, P. J. 1994. *C How to Program, 2nd Edition*. New York: Prentice Hall.
- Denny, P.; Smith IV, D. H.; Fowler, M.; Prather, J.; Becker, B. A.; and Leinonen, J. 2024. Explaining code with a purpose: An integrated approach for developing code comprehension and prompting skills. *arXiv preprint arXiv:2403.06050*.
- Garces, S.; Vieira, C.; Ravai, G.; and Magana, A. J. 2023. Engaging students in active exploration of programming worked examples. *Education and Information Technologies* 28:2869–2886.
- Graesser, A. C.; Wiemer-Hastings, P.; Wiemer-Hastings, K.; Harter, D.; Group, T. R. G. T. R.; and Person, N. 2000. Using latent semantic analysis to evaluate the contributions of students in autotutor. *Interactive Learning Environments* 8(2):129–147.
- Hosseini, R.; Akhuseyinoglu, K.; Brusilovsky, P.; Malmi, L.; Pollari-Malmi, K.; Schunn, C.; and Sirkiä, T. 2020. Improving engagement in program construction examples for learning python programming. *International Journal of Artificial Intelligence in Education* 30(2):299–336.
- Kelley, A., and Pohl, I. 1995. *C by Dissection : The Essentials of C Programming*. New York: Addison-Wesley.
- Koedinger, K. R.; Kim, J.; Jia, J. Z.; McLaughlin, E. A.; and Bier, N. L. 2015. Learning is not a spectator sport: Doing is better than watching for learning from a mooc. In *Proceedings of the second (2015) ACM conference on learning@ scale*, 111–120.
- Koutchene, C.; Dainese, N.; Sarsa, S.; Hellas, A.; Leinonen, J.; and Denny, P. 2024. Open source language models can provide feedback: Evaluating LLMs’ ability to help students using gpt-4-as-a-judge. In *Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1*, 52–58.
- Lekshmi-Narayanan, A.-B., and Brusilovsky, P. 2024. Evaluating correctness of student code explanations: Challenges and solutions. In Shi, Y.; Brusilovsky, P.; Akram, B.; Price, T.; Leinonen, J.; Koedinger, K.; and Lan, A., eds., *8th Educational Data Mining in Computer Science Education (CSEDM) Workshop at EDM2024*, volume 3796 of *CEUR Workshop Proceedings*. CEUR.
- Lekshmi-Narayanan, A.-B.; Oli, P.; Chapagain, J.; Hassany, M.; Banjade, R.; Brusilovsky, P.; and Rus, V. 2024. Explaining code examples in introductory programming courses: LLM vs humans. *Proceedings of Machine Learning Research* 257:107–117.
- Li, D.; Jiang, B.; Huang, L.; Beigi, A.; Zhao, C.; Tan, Z.; Bhattacharjee, A.; Jiang, Y.; Chen, C.; Wu, T.; et al. 2024. From generation to judgment: Opportunities and challenges of llm-as-a-judge. *arXiv preprint arXiv:2411.16594*.
- Linn, M. 1992. Can experts’ explanations help students develop program design skills. *International Journal on the Man-Machine Studies* 36:511–551.
- Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; and Stoyanov, V. 2019. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*.
- Oli, P.; Banjade, R.; Chapagain, J.; and Rus, V. 2024a. Automated assessment of students’ code comprehension using LLM. In Ananda, M.; Malick, D. B.; Burstein, J.; Liu, L. T.; Liu, Z.; Sharpnack, J.; Wang, Z.; and Wang, S., eds., *Proceedings of the 2024 AAAI Conference on Artificial Intelligence*, volume 257 of *Proceedings of Machine Learning Research*, 118–128. PMLR.
- Oli, P.; Banjade, R.; Chapagain, J.; and Rus, V. 2024b. Automated assessment of students’ code comprehension using LLM. In *AI for Education Workshop*, 118–128. PMLR.
- Oli, P.; Banjade, R.; Lekshmi Narayanan, A. B.; Brusilovsky, P.; and Rus, V. 2024c. Exploring the effectiveness of reading vs. tutoring for enhancing code comprehension for novices. In *Proceedings of the 39th ACM/SIGAPP symposium on applied computing*, 38–47.
- Reimers, N., and Gurevych, I. 2019. Sentence-bert: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992.
- Rus, V.; D’Mello, S.; Hu, X.; and Graesser, A. 2013a. Recent advances in conversational intelligent tutoring systems. *AI magazine* 34(3):42–54.
- Rus, V.; Lintean, M.; Banjade, R.; Niraula, N.; and Stefanescu, D. 2013b. SEMILAR: The semantic similarity toolkit. In Butt, M., and Hussain, S., eds., *Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 163–168.

- Sofia, Bulgaria: Association for Computational Linguistics.
- Rus, V.; Akhuseyinoglu, K.; Chapagain, J.; Tamang, L.; and Brusilovsky, P. 2021. Prompting for free self-explanations promotes better code comprehension. In *5th Educational Data Mining in Computer Science Education (CSEDM) Workshop at EDM2021*, volume 3051. CEUR.
- Sharrock, R.; Hamonic, E.; Hiron, M.; and Carlier, S. 2017. Codecast: An innovative technology to facilitate teaching and learning computer programming in a c language online course. In *Proceedings of the Fourth (2017) ACM Conference on Learning at Scale*, 147–148. ACM.
- Vihavainen, A.; Miller, C. S.; and Settle, A. 2015. Benefits of self-explanation in introductory programming. In *Proceedings of the 46th ACM Technical Symposium on Computer Science Education - SIGCSE '15*, 284–289. ACM Press.
- Wild, F.; Stahl, C.; Stermsek, G.; and Neumann, G. 2005. Parameters driving effectiveness of automated essay scoring with LSA. Technical report, Loughborough University.
- Zhang, Q.; Wang, Y.; Yu, T.; Jiang, Y.; Wu, C.; Li, L.; Wang, Y.; Jiang, X.; Shang, L.; Tang, R.; et al. 2024. Revisal: Improving llm-as-a-judge via response-adapted references. *arXiv preprint arXiv:2410.05193*.
- Zhou, Y.; Muresanu, A. I.; Han, Z.; Paster, K.; Pitis, S.; Chan, H.; and Ba, J. 2020. Large language models are human-level prompt engineers. In *The eleventh international conference on learning representations*.